Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date:**	**Enrolment No:92301733021**

Aim: To understand the constraint satisfaction problem in problem formulation

IDE: Google Colab

Theory:

The Constraint Satisfaction Problem (CSP) is a fundamental concept in computer science and artificial intelligence, playing a crucial role in solving a wide range of real-world problems. At its core, CSP involves finding a solution to a problem where the solution must satisfy a set of constraints that define the problem's requirements and limitations. These constraints restrict the possible values that variables can take, leading to a search for a combination of values that satisfies all constraints simultaneously.

CSPs can be formulated in various domains, including scheduling, planning, resource allocation, configuration, and design. Examples of CSP applications include job scheduling, timetabling, Sudoku puzzles, map coloring, and the traveling salesman problem. In each case, the problem consists of a set of variables, each with a domain of possible values, and a set of constraints that specify allowable combinations of variable values.

The basic components of a CSP include:

1. **Variables**: These represent the entities whose values need to be determined. Each variable has a domain, which is a set of possible values it can take.
2. **Domains**: A domain is the set of values that a variable can take. Domains can be finite or infinite, discrete or continuous, depending on the problem domain.
3. **Constraints**: Constraints define the relationships among variables and restrict the combinations of values they can take. Constraints can be unary (applying to a single variable), binary (between two variables), or higher-order (involving three or more variables).

The goal in solving a CSP is to find an assignment of values to variables such that all constraints are satisfied. This involves searching through the space of possible assignments to find a solution or determine that no solution exists. The search process typically involves backtracking, constraint propagation, and variable ordering heuristics to efficiently explore the search space.

One common approach to solving CSPs is backtracking search, which systematically explores the space of possible assignments by making choices and backtracking when a dead-end is reached. Backtracking search is guided by variable and value ordering heuristics to prioritize the search for promising solutions and avoid exploring unpromising regions of the search space.

Another technique used in CSP solving is constraint propagation, which involves enforcing constraints locally to reduce the domain of variables and prune the search space. Constraint propagation techniques include arc consistency, domain reduction, and constraint propagation algorithms such as AC-3 and AC-4.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date:**	**Enrolment No:92301733021**

In addition to backtracking search and constraint propagation, other methods for solving CSPs include local search algorithms, genetic algorithms, and constraint satisfaction neural networks. Each approach has its strengths and weaknesses, making them suitable for different types of CSPs and problem domains.

Despite the challenges involved in solving CSPs, they offer a powerful framework for modeling and solving complex combinatorial optimization problems. By formulating real-world problems as CSPs and applying appropriate solving techniques, researchers and practitioners can tackle a wide range of practical problems efficiently and effectively.

Methodology:

1. **Mention the constraints for sudoku solution**

A valid Sudoku solution must satisfy these constraints:

- **Row Constraint**: Each number (1–9) must appear only once in every row.
- **Column Constraint**: Each number (1–9) must appear only once in every column.
- **Subgrid Constraint**: Each number (1–9) must appear only once in each 3×3 subgrid.
- **Domain Constraint**: Each cell can take values only from 1 to 9.
- **Pre-filled Constraint**: Initially given numbers cannot be changed.

2. **Solve the sudoku using the CSP approach**

In CSP, Sudoku is solved by defining:

- **Variables** → Each cell in the grid
- **Domain** → Values {1,2,3,4,5,6,7,8,9}
- **Constraints** → Row, column, and subgrid rules

Steps to Solve

1. Initialize the grid with given values.
2. Select an unassigned cell (variable).
3. Check domain values (1–9) for that cell.
4. Apply constraints:
   - Value should not repeat in row, column, or subgrid.
5. Assign valid value.
6. Move to next cell (Backtracking).
7. If no valid value → Backtrack and try another value.
8. Continue until all cells are filled.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date:**	**Enrolment No:92301733021**

Program (Code):

To be attached with

```
# Function to check if it is safe to place num at mat[row][col]
def isSafe(mat, row, col, num):

    # Check if num exists in the row
    for x in range(9):
        if mat[row][x] == num:
            return False

    # Check if num exists in the col
    for x in range(9):
        if mat[x][col] == num:
            return False

    # Check if num exists in the 3x3 sub-matrix
    startRow = row - (row % 3)
    startCol = col - (col % 3)

    for i in range(3):
        for j in range(3):
            if mat[i + startRow][j + startCol] == num:
                return False

    return True

# Function to solve the Sudoku problem
def solveSudokuRec(mat, row, col):
    # base case: Reached nth column of the last row
    if row == 8 and col == 9:
        return True

    # If last column of the row go to the next row
    if col == 9:
        row += 1
        col = 0

    # If cell is already occupied then move forward
    if mat[row][col] != 0:
        return solveSudokuRec(mat, row, col + 1)

    for num in range(1, 10):
```

```
        # If it is safe to place num at current position
        if isSafe(mat, row, col, num):
            mat[row][col] = num
            if solveSudokuRec(mat, row, col + 1):
                return True
            mat[row][col] = 0

    return False

def solveSudoku(mat):
    solveSudokuRec(mat, 0, 0)

if __name___== "__main__":
    mat = [
        [3, 0, 6, 5, 0, 8, 4, 0, 0],
        [5, 2, 0, 0, 0, 0, 0, 0, 0],
        [0, 8, 7, 0, 0, 0, 0, 3, 1],
        [0, 0, 3, 0, 1, 0, 0, 8, 0],
        [9, 0, 0, 8, 6, 3, 0, 0, 5],
        [0, 5, 0, 0, 9, 0, 6, 0, 0],
        [1, 3, 0, 0, 0, 0, 2, 5, 0],
        [0, 0, 0, 0, 0, 0, 0, 7, 4],
        [0, 0, 5, 2, 0, 6, 3, 0, 0]
    ]

    solveSudoku(mat)

    for row in mat:
        print(" ".join(map(str, row)))
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date:**	**Enrolment No:92301733021**

Results:

To be attached with Solved sudoku

```
•••  3 1 6 5 7 8 4 9 2
     5 2 9 1 3 4 7 6 8
     4 8 7 6 2 9 5 3 1
     2 6 3 4 1 5 9 8 7
     9 7 4 8 6 3 1 2 5
     8 5 1 7 9 2 6 4 3
     1 3 8 9 4 7 2 5 6
     6 9 2 3 5 1 8 7 4
     7 4 5 2 8 6 3 1 9
```

Observation and Result Analysis:

- The program uses backtracking to solve Sudoku. It tries numbers in empty cells, checks if they follow the rules (row, column, box), and backtracks if something goes wrong.
- The puzzle is solved correctly with all constraints satisfied. The method is reliable and gives a valid solution, though it may take more time for harder puzzles.

Post Lab Exercise:

Write the code and output for N-Queens problem. Take the value of N from the user.

Code:

```
def is_safe(board, row, col, n):
    # Check column
    for i in range(row):
        if board[i][col] == 1:
            return False
    # Check left diagonal
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    # Check right diagonal
    i, j = row - 1, col + 1
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
```

```
        i -= 1
        j += 1
    return True
def solve_n_queens(board, row, n):
    if row == n:
        return True
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1
            if solve_n_queens(board, row + 1, n):
                return True
            board[row][col] = 0  # backtrack
    return False

def print_board(board, n):
    for i in range(n):
        for j in range(n):
            if board[i][j] == 1:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
# Main
n = int(input("Enter value of N: "))
board = [[0 for _ in range(n)] for _ in range(n)]

if solve_n_queens(board, 0, n):
    print("\nSolution:")
    print_board(board, n)
else:
    print("No solution exists")
```

Output:

```
Enter value of N: 9

Solution:
Q . . . . . . . .
. . Q . . . . . .
. . . . . Q . . .
. . . . . . . Q .
. Q . . . . . . .
. . . Q . . . . .
. . . . . . . . Q
. . . . . . Q . .
. . . . Q . . . .
```